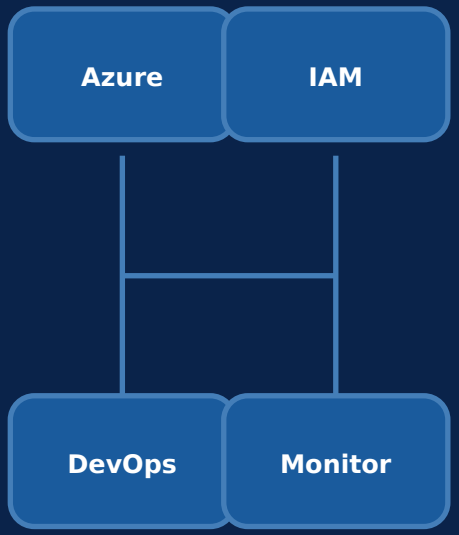
AZURE • DEVOPS • DEVSECOPS

CLOUDGUARD 360

Architecture Explained

Network Architecture + Azure Cloud Architecture
A step-by-step implementation handbook



Terraform / Entra ID / Azure RBAC / Docker / AKS
GitHub Actions / Zabbix / Grafana / Incident Automation

Educational project architecture • Not a production deployment specification

Includes exact reference diagrams, component walkthroughs, build stages and cost controls

01 | Read the two diagrams together

Two complementary perspectives of the same CloudGuard 360 platform.

NETWORK ARCHITECTURE	AZURE CLOUD / DEVSECOPS ARCHITECTURE
Shows where the networks and workloads live.	Shows how software moves from code to deployment and support.
Explains VNet, subnet, peering, routing, NSG and optional hybrid connectivity.	Explains Git, CI/CD, Terraform, Entra ID, RBAC, AKS and operational workflows.
Answers: **How can components connect safely?**	Answers: **How do we build, secure and operate the application?**
Primary concern: connectivity, segmentation and traffic control.	Primary concern: lifecycle, automation, identity and reliability.

One project, three layers of responsibility

Layer	Purpose	Core components
1 — Infrastructure	Where resources are hosted and how packets travel.	Azure VNets, subnets, NSGs, VNet peering; optional AKS nodes and AD DS VM.
2 — Software delivery	How infrastructure and application changes are built and released.	GitHub Actions, Terraform, Docker images, registry, Kubernetes and GitOps.
3 — Identity & operations	Who can act, how health is observed and how failures are handled.	Microsoft Entra ID, Azure RBAC, Zabbix, Grafana and approved recovery workflows.

Architecture status: The diagrams show a **target-state learning design**, not proof that all resources already exist. Dashed/optional components may require licences, billable Azure services or dedicated laboratory setup.

02 | Network Architecture — Reference Diagram

Hub-and-Spoke Azure networking, segmentation and optional identity/operations extensions.

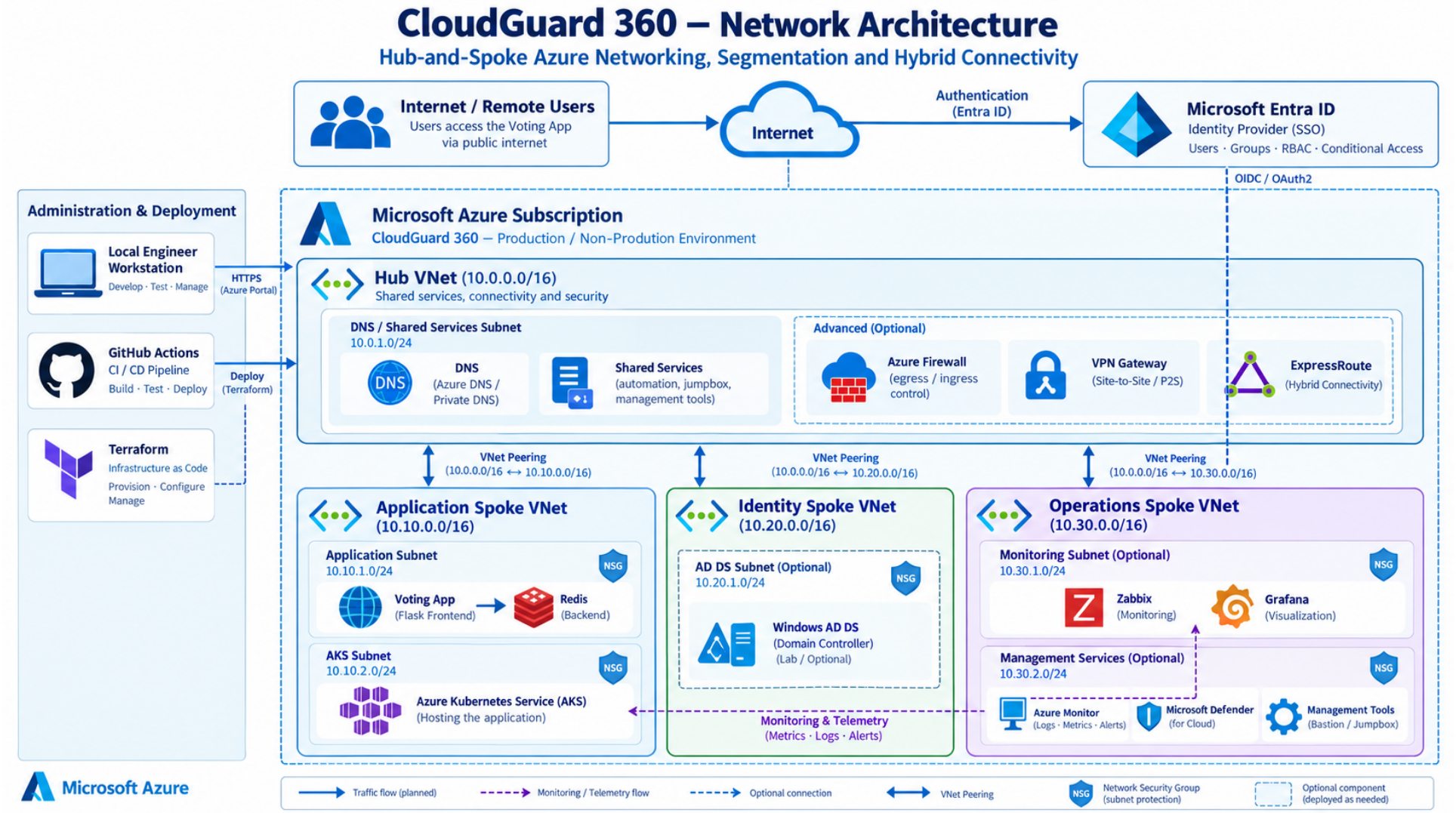


Figure 1. Original CloudGuard 360 network architecture PNG. Zoom into the PDF for smaller technical labels. CIDR addresses are illustrative and must be checked for overlap before deployment.

03 | Explain the network diagram

The Hub is the shared network; the Spokes separate application, identity and operations workloads.

Zone / component	Meaning	Planned implementation
Hub VNet 10.0.0.0/16	Shared network foundation: DNS and, if needed, administration and hybrid services.	Real VNet + peering. Azure Firewall/VPN/ExpressRoute are **optional**.
Application Spoke 10.10.0.0/16	Dedicated environment for application services and optional AKS nodes.	VNet, subnet and NSG. Voting App runs locally first; AKS is a temporary exercise.
Identity Spoke 10.20.0.0/16	Isolated place for a Windows AD DS training domain controller.	Design initially; deploy VM only for the AD laboratory.
Operations Spoke 10.30.0.0/16	Optional cloud home for monitoring or management components.	Zabbix and Grafana run **locally by default** to reduce costs.
VNet Peering	Private connectivity between the Hub and each selected Spoke.	Create and verify. Peering is not automatically transitive.
NSG & routes	Permit or deny network flows and choose forwarding paths.	Test effective rules and routes; never point UDRs to a nonexistent appliance.
Entra ID	Cloud identity service for users and workloads.	Authentication integration is separate from the network path.

What the IP ranges mean

For example, **10.10.0.0/16** is a private IPv4 address range assigned to the Application VNet. A smaller subnet, such as **10.10.1.0/24**, fits inside it. Azure reserves some addresses within each subnet. CIDR ranges must not overlap when peering or routing between networks.

Critical networking distinction: A Hub VNet does not automatically provide firewall inspection. Without a deployed routing appliance or Azure Firewall and proper UDRs, it is only a network topology with shared services.

04 | Follow a request through the network

Understand the data path rather than memorising service names.

Scenario A — A user opens the Voting App

Step	What happens	Engineer checks
1 — Browser	The user opens the configured application address.	DNS answer and target endpoint.
2 — Sign-in	If the optional OIDC proxy is enabled, the user is redirected to Microsoft Entra ID.	Application registration, redirect URI, login result.
3 — Ingress	The configured ingress, proxy, port-forward or service accepts the request.	Public vs private exposure, TLS and endpoint health.
4 — Application	The Flask frontend runs in a Docker container or Kubernetes Pod.	Pod/container health, logs and service routing.
5 — Data	The frontend communicates with the Redis backend.	Service discovery, port reachability and backend health.

Scenario B — A packet cannot reach another network

- **Start with DNS:** is the intended name resolved to the correct address?
- **Inspect effective routes:** is the next hop valid in both directions?
- **Check security:** NSG, relevant platform rules and host firewall.
- **Check the service:** is the target process listening and healthy?
- **Verify connectivity:** use controlled tests and evidence rather than removing protections blindly.

Private Endpoints: A Private Endpoint uses an IP in a VNet for access to a supported service. Private DNS must resolve the relevant name appropriately; deploying a Private Endpoint does not, by itself, guarantee public access is disabled.

05 | Azure Cloud Architecture — Reference Diagram

Secure platform engineering, infrastructure automation, software delivery and observability.

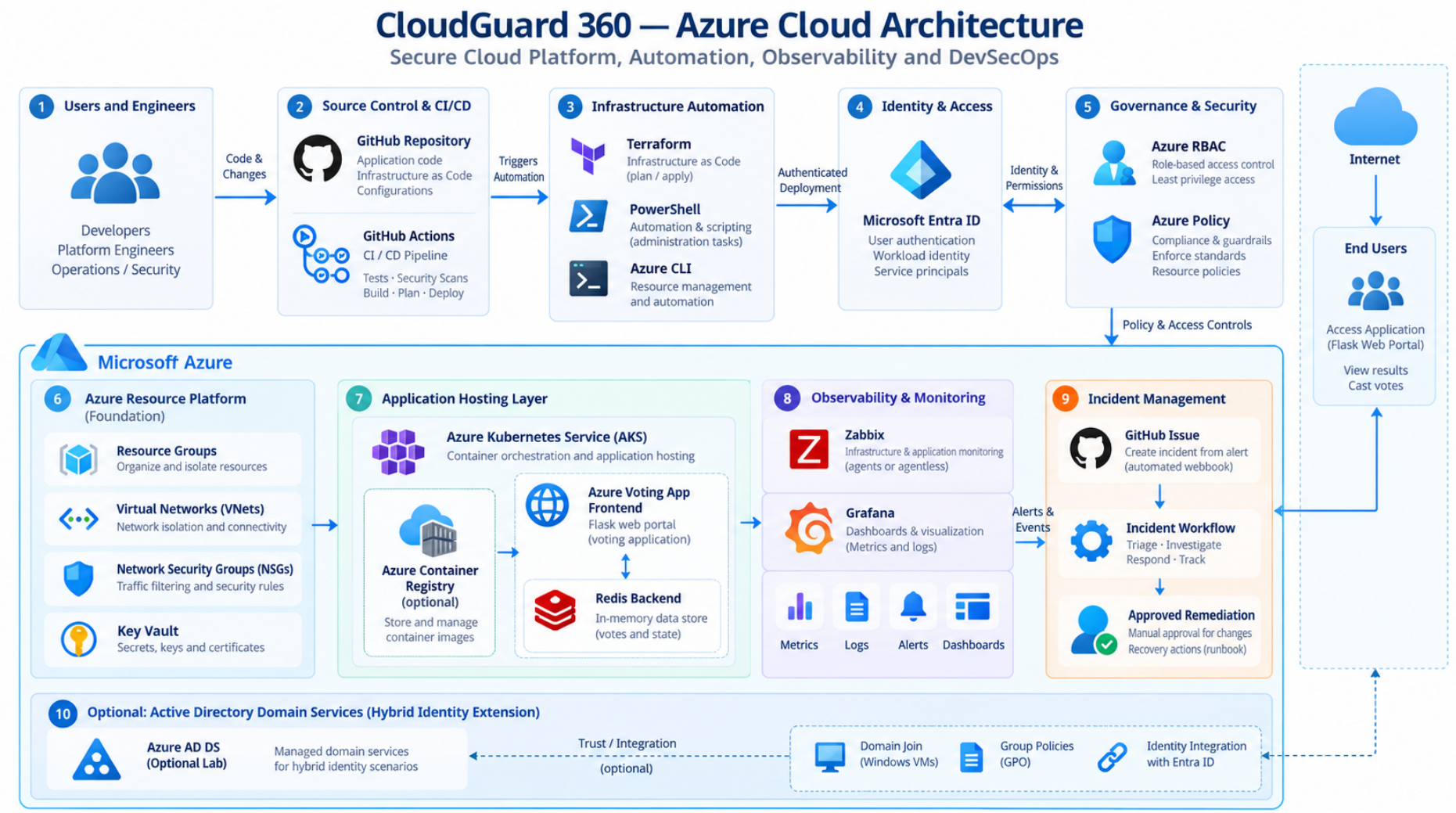


Figure 2. Original Azure cloud architecture PNG. The AD DS identity extension is optional and should be validated separately.

06 | Explain the Azure Cloud diagram

Follow one change from GitHub to a running, monitored application.

Stage	Technology	Responsibility in the project
01 — Source control	Git + GitHub	Tracks application files, infrastructure definitions, reviews and changes.
02 — CI / security	GitHub Actions	Runs tests, SAST/SCA, secret checks, image scans and build workflows.
03 — Infrastructure	Terraform	Provisions Azure Resource Groups, VNets, NSGs and optional AKS resources.
04 — Administration	PowerShell + Azure CLI	Queries environments, creates inventories and supports controlled maintenance.
05 — Identity	Microsoft Entra ID	Authenticates humans and federated pipeline identities; optional app OIDC.
06 — Authorisation	Azure RBAC + Policy	Limits resource administration and applies supported configuration guardrails.
07 — Runtime	Docker + AKS (optional)	Packages Flask and Redis; orchestrates Pods when AKS is deployed.
08 — Observation	Zabbix + Grafana	Collects availability/telemetry and visualises incidents and trends.
09 — Response	GitHub Issues + runbooks	Tracks detected events, investigation, approval and post-recovery checks.

Clarification: An **Azure RBAC role** grants control-plane permissions to Azure resources. **Entra application sign-in** authenticates users, but application authorisation requires its own enforcement (e.g., proxy admission restrictions or backend checks).

07 | The end-to-end delivery flow

This is the complete CloudGuard 360 software factory, in the right order.

Sequence	Action	Expected evidence
1 — Change	Engineer opens a branch and Pull Request.	Repository commit and reviewed PR.
2 — Validate	GitHub Actions runs tests, code/secret/dependency checks.	Pipeline results and scan reports.
3 — Package	Docker builds the frontend image; Redis is a separate service.	Versioned image reference and reproducible build.
4 — Provenance	Generate an SBOM, scan and optionally sign/attest the image.	SBOM and verification results.
5 — Provision	Terraform validates/plans/creates Azure resources.	Plan and applied infrastructure records.
6 — Authenticate	GitHub Actions receives an OIDC-based workload token if configured.	Federated credential and scoped role assignment.
7 — Deploy	Use kind locally, then temporary AKS with controlled exposure.	Pods healthy, Service reachable, readiness checks.
8 — Observe	Zabbix checks health; Grafana shows the relevant metrics.	Events and dashboard panels.
9 — Respond	Create an incident record, investigate, approve corrective action.	GitHub issue, runbook and recovery evidence.

Application selection: Microsoft Azure Voting App (Flask frontend + Redis backend). We reuse an existing sample rather than build business logic from scratch. Enterprise authentication and security are additional project work, not native features of the sample.

08 | Implementation roadmap — Foundation

Follow these stages sequentially. Test each stage before moving to the next.

Stage	Step-by-step work	Pass criterion
01 — Environment	Check Azure Free Trial credit/expiry; select subscription; prepare Cloud Shell, Git, Docker, Terraform, VS Code. Set an Azure budget and alerts.	Correct subscription identified; spending controls documented.
02 — Starter app	Fork the Azure Voting App, clone the repo, review dependencies and Docker Compose, run locally.	Vote is recorded locally; containers healthy.
03 — GitHub repo	Create documented folders: docs/, app/, infrastructure/, kubernetes/, observability/, security/, .github/. Add .gitignore and branch rules.	Version-controlled repository; no secrets or state in Git.
04 — Azure IaC	Write reusable Terraform modules for Resource Groups, VNet, subnets, peering and NSGs. Run fmt, init, validate, plan, review and apply.	Created resources match the design; second plan shows no unintended drift.

Minimal commands to confirm the starting point

Tool	Command	Why
Azure CLI	**az account show --output table**	Inspect the selected subscription.
Docker	**docker compose up --build -d**	Build and run the sample application locally.
Terraform	**terraform validate**	Check Terraform configuration validity after init.
Git	**git status**	Review tracked changes before commits.

Stop/go gate: Do not provision AKS yet. First prove that the app runs locally and that Terraform can create and safely destroy a minimal network environment.

09 | Implementation roadmap — Identity & deployment

Now add permissions, scripted administration and a deployment workflow.

Stage	Step-by-step work	Pass criterion
05 — IAM	Create Entra lab groups; assign Azure RBAC at Resource Group scope; test permitted and denied actions. Configure GitHub OIDC federation with constrained trust.	Reader cannot modify resources; pipeline identity has bounded rights.
06 — Automation	Use Azure CLI and PowerShell (Az module) to inventory resources, routes, tags and role assignments. Export sanitised reports.	Repeatable, readable inventory; no sensitive tokens in output.
07 — CI/CD	Configure Pull Request workflow, unit tests, secret scan, SAST/SCA, Docker build and image scan. Optionally publish a verified image.	Failing check blocks deployment; fixed PR passes.
08 — Kubernetes	Run kind locally, then create AKS for a short lab only. Deploy the app; inspect Pods, Services, events and probes. Delete AKS after the exercise.	Service works; operator can explain kubectl diagnostics and cleanup.

Optional identity exercise — AD DS

To practise traditional Active Directory, use a separately scoped Windows Server AD DS lab on a suitable temporary VM or local virtualisation. Configure DNS, OUs, users, groups, GPO and a diagnostic runbook. A domain controller in Azure is not automatically a genuine hybrid on-premises-to-cloud topology; synchronisation must be designed and tested separately.

AKS pricing reality: AKS Free tier management does not make its compute nodes, storage, IPs or network traffic free. Do not assume the free-trial B1s allowance satisfies AKS system node requirements.

10 | Implementation roadmap — Operate & assure

Introduce production-style monitoring, response, security evidence and a final audit.

Stage	Step-by-step work	Pass criterion
09 — Monitoring	Run Zabbix + Grafana via local Docker Compose. Configure HTTP health checks and Zabbix/Grafana integration.	An application outage generates a Zabbix event and dashboard change.
10 — NetworkOps	Use approved Aruba Central access, or transparently simulate three branches with latency, availability, AP and VLAN data.	Network event can be correlated; simulator labelled as synthetic.
11 — Incident workflow	Send Zabbix alert to GitHub Issue. Investigate health/logs. Require approval for selected recovery steps; verify restoration.	One complete timeline: detect → ticket → diagnose → approve → recover.
12 — DevSecOps	Add IaC checks, SBOM, image analysis, policy-as-code and optional Argo CD reconciliation. Record exceptions.	A deliberately non-compliant test configuration is caught, then fixed.
13 — Final validation	Test controls, review architecture choices, run cleanup, document costs and write findings, evidence and recommendations.	Another engineer can reproduce documented lab stages and limitations.

Operational principle: Do not automatically restart production services solely on an HTTP check failure. The project first uses **human-approved**, scoped recovery actions with pre/post validation and an audit trail.

11 | Incident-response exercise — One complete story

A safe, measurable failure-and-recovery scenario that connects monitoring, Azure and DevOps.

Moment	What happens	Evidence to capture
T0 — Normal	Voting App responds to its configured health check.	Baseline Zabbix metric and Grafana screenshot.
T1 — Failure	Intentionally stop the lab application or scale replicas to zero.	Change record and timestamp.
T2 — Detect	Zabbix trigger changes state to problem.	Trigger history, HTTP/availability readings.
T3 — Ticket	A webhook creates an incident GitHub Issue.	Issue identifier and payload summary.
T4 — Investigate	Collect Pod status, events, container logs and relevant network checks.	Diagnostics, hypotheses and root cause.
T5 — Approve	Operator authorises a narrowly scoped restoration.	Approval and authorised identity.
T6 — Restore	Restore the known-good deployment or correct its replica count.	Action log, version and deployment status.
T7 — Verify	Health checks recover; monitoring event closes.	Recovery confirmation and incident report.

Example diagnostic commands (replace placeholders)

Purpose	Command
Pods	**kubectl get pods -n demo**
Detailed failure	**kubectl describe deployment voting-app -n demo**
Events	**kubectl get events -n demo**
Rollback (only if revision exists)	**kubectl rollout undo deployment/voting-app -n demo**

Commands are illustrative: verify the actual namespace and Deployment name before use. A rollback requires a suitable previous revision; scaling a Deployment back up is the appropriate response for some simulations.

12 | Cost and security guardrails

Make the platform technically convincing without exhausting Azure Free Trial credit.

Resource / feature	Low-cost approach	Important caveat
Voting App, Docker, kind	Run locally first.	Host must remain running for access and monitoring.
Zabbix + Grafana	Use a local Docker Compose profile.	Monitor cannot automatically access a private Azure workload.
Azure networking	Provision VNets, NSGs and peering only as needed.	Peering traffic and supporting services may be billed.
Entra ID / Azure RBAC	Use available free tenant features and minimal roles.	Conditional Access/PIM may require premium licences.
Terraform	Use CLI and small modules; protect local state before remote backend.	Backend storage may have charges; secrets can appear in state.
AKS	Create only for an explicit, short exercise and tear down fully.	Nodes, disks, egress and load balancing are billable.
AD DS	Temporary appropriately sized VM or local lab.	Windows/licensing/VM/storage charges may apply.
Firewall / VPN / ExpressRoute	Architecture study until a cost-approved session.	Often unsuitable as continuously running trial resources.

Operating rules

- **Budget first:** verify subscription type, credit, region, quotas and the current price estimate before every major deployment.
- **Least privilege:** no subscription Owner role for routine CI/CD. Limit federated trust and Azure scopes.
- **No public secrets:** keep credentials, Terraform state, access tokens and real internal data out of Git.
- **Safe exposure:** prefer port-forward or restricted test ingress over unneeded public LoadBalancers.
- **Cleanup after each lab:** verify no leftover disks, public IPs, monitoring ingestion or cluster components remain.

13 | Completion checklist and portfolio narrative

A finished laboratory is proven by evidence, not by diagrams alone.

Proof area	What you should be able to show
Azure foundation	Resource inventory, current topology, correct address ranges, role assignments and cost report.
Networking	Effective routes, NSG rule results, permitted and denied connectivity tests.
Identity	User sign-in where configured, scoped service identity and denied unauthorised action.
Infrastructure as Code	Terraform plan, successful apply, drift check and controlled destroy.
Application & Kubernetes	Working Voting App, healthy Pods, Service exposure model and rollback evidence.
CI/CD & DevSecOps	Pull Request history, tests, vulnerability scan, SBOM and policy-check results.
Observability	Zabbix incident, Grafana trends and monitoring configuration.
Incident response	Issue record, investigation notes, authorisation, remediation and health verification.
Governance	Risk register, residual gaps, expense summary, limitations and next-step roadmap.

A 60-second technical explanation for an interview

“I built CloudGuard 360 as an end-to-end Azure engineering laboratory. I designed a Hub-and-Spoke target network and deployed its foundational components with Terraform. I used Microsoft Entra ID and scoped Azure RBAC for identity and administration, GitHub Actions for build and security checks, and Docker with local Kubernetes before a temporary AKS deployment. Zabbix and Grafana provided service monitoring. I then simulated a failure, tracked the incident and demonstrated an approved recovery with evidence. Throughout, I documented which controls were fully tested, simulated or optional, and kept Azure costs under review.”

Final acceptance criterion: An independent reviewer can follow the documentation to reproduce the supported lab stages, understand every critical design choice, find the evidence and clearly identify what has **not** been deployed or verified.

14 | Useful links, glossary and next actions

Official starting points for deployment, identity and security learning.

Reference	Location
Project repository package	CloudGuard360_GitHub_Complete_Project.zip (provided alongside this handbook)
Azure Voting App sample	https://github.com/Azure-Samples/azure-voting-app-redis
Azure Cloud Shell	https://shell.azure.com/
Azure pricing calculator	https://azure.microsoft.com/pricing/calculator/
Azure landing zone design	https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/
Azure architecture hub and spoke	https://learn.microsoft.com/azure/architecture/networking/architecture/hub-spoke
AKS documentation	https://learn.microsoft.com/azure/aks/
Terraform documentation	https://developer.hashicorp.com/terraform/docs
Zabbix documentation	https://www.zabbix.com/documentation/current/en/manual
Grafana Zabbix plugin	https://grafana.com/grafana/plugins/alexanderzobnin-zabbix-app/

Mini-glossary

Term	Meaning	Term	Meaning
VNet	Azure private virtual network	NSG	Network security rules
UDR	User-defined route	VNet Peering	Private connection between VNets
OIDC	OpenID Connect federation	Azure RBAC	Azure resource authorisation
AKS	Azure Kubernetes Service	IaC	Infrastructure as Code
SBOM	Software Bill of Materials	SLI / SLO	Measured indicator / reliability target